

Homework 3 — Multi-Process Word Transportation and Concurrent Sorting System

GTU CSE344 System Programming

Dicle Çoban

April 15, 2026

Cover Information

Course	CSE344 — System Programming
Homework	Homework 3 — Multi-Process Word Transportation and Concurrent Sorting System
Submission Date	2026-04-15
Student	Dicle Çoban
Language	C (POSIX-compatible, multi-process implementation)

1 Introduction

This homework implements a multi-process word transportation and concurrent sorting system in C on a POSIX-compatible environment. The system reads words from an input text file, admits them to arrival floors, transports their characters one by one to sorting floors, and reconstructs the original words concurrently.

The goal of the homework is not only to move data from one place to another, but to manage concurrent access to shared data structures correctly. Multiple independent processes may attempt to claim words, claim character tasks, update sorting areas, and use elevator subsystems at the same time. Therefore, the main design challenges are:

- safe shared-memory usage,
- synchronization of all concurrent updates,
- prevention of duplicate work,
- atomic floor-capacity admission,
- coordinated completion detection,
- and clean process termination.

2 System Architecture

2.1 Overall Architecture

The implementation is centered around one parent/coordinator process and multiple child processes that cooperate through a shared memory region created with `mmap(..., MAP_SHARED | MAP_ANONYMOUS, ...)`. The system consists of six active subsystems:

- **Parent / Coordinator Process:** parses arguments, reads input, initializes shared state and synchronization objects, spawns all child processes, monitors completion, writes the final output file, prints summary information, and performs shutdown.
- **Word-Carrier Processes:** fixed to their initial floor; scan the input list in synchronized round-robin order and admit words only when capacity rules are satisfied.
- **Letter-Carrier Processes:** mobile worker processes; claim one character at a time and either directly place it or request elevator transportation.
- **Sorting Processes:** fixed to a sorting floor; incrementally reconstruct words using sorting metadata, fixed positions, moves, and swaps.
- **Delivery Elevator Process:** handles character transportation requests.
- **Reposition Elevator Process:** handles relocation requests of idle letter-carrier processes.

2.2 Process Responsibilities

Process Type	Responsibility
Parent / Coordinator	Initializes the system, creates all child processes, monitors completion, generates output, prints final statistics, handles shutdown and cleanup.
Word-Carrier	Scans the input list in round-robin order, atomically claims a candidate word, checks arrival-floor and sorting-floor capacity, admits or releases the word.
Letter-Carrier	Claims one character task from the current floor, uses direct placement or delivery elevator, remains at destination floor after delivery, requests reposition when idle.
Sorting Process	Works on words assigned to its own floor, fixes correct positions, moves misplaced characters, performs swaps, and marks words as completed.
Delivery Elevator	Serves character delivery requests through the delivery queue and wakes the corresponding letter-carrier after delivery completes.
Reposition Elevator	Serves idle-carrier movement requests through the reposition queue and wakes the corresponding letter-carrier after relocation.

3 Data Structures and Shared Memory Design

3.1 Shared State Overview

All process-visible state is placed inside a single shared `SharedState` structure. This structure contains:

- global configuration and counters,
- all words loaded from the input file,
- per-floor information,
- per-carrier state,
- per-sorter and per-word-carrier statistics,
- delivery and reposition request queues,
- and all synchronization primitives used between processes.

3.2 SharedWord Structure

Each word is represented by a `SharedWord` structure. It stores:

- `word_id`,
- original text,
- `sorting_floor`,
- `arrival_floor`,
- claimed, admitted, and completed flags,
- `sorting_area[]`,
- `sorting_meta[]`,
- `occupied[]`,
- `fixed[]`,
- per-character state arrays such as `char_claimed[]`, `char_delivered[]`, `char_in_transit[]`, and `char_arrived[]`.

The `sorting_meta[]` array stores the original index of the character currently occupying a sorting slot. This is the key structure that allows correct reconstruction, including repeated characters.

3.3 Other Shared Structures

Structure	Purpose
FloorState	Stores the current number of active words and the current letter-carrier count on each floor.
CarrierState	Stores a carrier's current floor, statistics, and private event semaphore information used to block and wake that carrier.
DeliveryRequest	Represents a pending character delivery request: source floor, destination floor, word index, character index, and carrier slot.
RepositionRequest	Represents a pending reposition request for an idle carrier.

4 Synchronization Strategy

The implementation uses POSIX named semaphores and shared-memory synchronization. The main synchronization objects are:

- `global_mutex`: protects globally shared metadata such as the round-robin cursor and slot allocation.
- Per-floor mutexes: protect `active_words` and `carrier_count`.
- Per-word mutexes: protect claim flags, delivery state, sorting area updates, completion flags, and word-local consistency.
- `sorter_lock` per word: ensures only one sorting process works on a specific word at a time.
- `delivery_mutex` and `reposition_mutex`: protect the two elevator request queues.
- `delivery_items` and `reposition_items`: counting semaphores used to wake elevator processes when queued work exists.
- `parent_event`: wakes the parent when important global events occur.
- `word_available_sem`: used by word-carrier processes when they must wait for future admission opportunities.
- Private carrier event semaphores: used by elevator processes to wake the correct letter-carrier after delivery or reposition.

4.1 Race Condition Prevention

Race conditions are prevented by combining shared memory with explicit critical sections:

- word claiming is synchronized under the global mutex,
- admission checks lock the relevant floor mutexes in increasing floor order,
- character-task claiming is protected by the word mutex,
- sorting-area updates are also protected by the word mutex,
- same-word concurrent sorting is blocked by the per-word sorter lock.

4.2 Critical Sections

The most important critical sections are:

- round-robin cursor update and word claim,
- floor-capacity admission and reservation,
- character-task claim / delivery state update,
- sorting-area move or swap operations,
- word completion and floor-capacity release,
- queue insertion for delivery and reposition requests.

5 Word Admission Logic

Word-carrier processes implement synchronized round-robin scanning. A shared round-robin cursor points to the next candidate position in the input list. When a word-carrier starts a scan:

1. it acquires the global mutex,
2. it searches from the current cursor position for an unclaimed, not-yet-admitted, not-yet-completed word,
3. it marks that word as claimed atomically,
4. it advances the shared cursor,
5. it releases the global mutex.

Then the process checks admission. Admission is accepted only if:

- the arrival floor has capacity,
- the sorting floor has capacity.

This check is performed atomically by locking the two floor mutexes in a deterministic order. If both checks succeed, the capacities are reserved immediately and the word becomes admitted. If either check fails, the word is released and may be selected again later. This implements the all-or-nothing admission policy required by the homework.

6 Character Transportation and Elevators

6.1 Character Tasks

Each word is decomposed implicitly by using the original text and per-character arrays. A letter-carrier process scans admitted words on its current floor and selects exactly one character such that:

- it is not already claimed,
- it is not already delivered,
- it is not already in transit.

When such a character is found, the carrier marks it as claimed and in transit under the word mutex.

6.2 Delivery Rules

If the sorting floor is equal to the carrier's current floor, the carrier directly places the character into the word's sorting area. Otherwise, it creates a delivery request and waits on its private event semaphore until the delivery elevator completes the request.

After a successful delivery:

- the character is placed into the sorting area,
- its delivery flag is updated,
- the carrier's current floor becomes the destination floor,
- the carrier continues execution from that floor.

6.3 Delivery Elevator

The delivery elevator maintains its own independent request queue. It wakes when delivery requests exist, collects a batch up to its configured capacity, processes the requests, updates delivery statistics, and signals the corresponding carriers after each request is completed.

6.4 Reposition Elevator

If a letter-carrier finds no available task on its current floor, it requests reposition to a random floor. The reposition elevator handles these requests through a separate queue and separate synchronization primitives. After relocation, the carrier resumes execution from its new floor.

6.5 Idle-Carrier Recovery

The parent process also monitors floors with zero active letter-carriers. If a floor becomes empty and the system is not yet complete, the parent respawns the configured number of letter-carriers for that floor. This prevents total disappearance of workers from a floor.

7 Sorting Algorithm

Characters are not inserted directly to their final positions. Instead, each delivered character is first placed into the first suitable non-fixed empty slot in `sorting_area[]`. The corresponding `sorting_meta[]` entry records the original index of that character in the word.

The sorting process works as follows:

- If the slot is empty, it does nothing.
- If the slot is fixed, it does nothing.
- If the character's original index matches the current slot index, it marks that slot as fixed.
- If the target slot is empty, it moves the character there.
- If the target slot is occupied and not fixed, it swaps the two entries.
- If the target slot is fixed, it leaves the character unchanged for a future pass.

This design satisfies the required move/swap rules. Because each character keeps its original index through `sorting_meta[]`, repeated letters are handled correctly and do not become ambiguous.

8 Termination Detection

Word completion is detected by checking whether all entries of `fixed[]` are equal to 1. When a word completes:

- its `completed` flag is set,
- the global completed-word counter is incremented,
- the reserved capacity on arrival and sorting floors is released,
- a wake-up is sent to the parent if all words are complete.

The parent waits on `parent_event` while monitoring `completed_words`. Once all words are complete, the parent:

1. sets the global shutdown flag,
2. wakes blocked processes,
3. creates the final output file,
4. prints summary statistics,
5. waits for all child processes,
6. releases resources.

The program also installs `SIGINT` and `SIGTERM` handlers so that Ctrl+C triggers a controlled shutdown path.

9 Output File Generation

Output verification: After each execution, the produced output file was compared with the corresponding input file. The verification process checked four conditions: (1) every input word appeared exactly once in the output, (2) the original `sorting_floor` value of each word was preserved, (3) the output file contained no blank or malformed lines, and (4) the final ordering matched the assignment specification, namely sorting first by `sorting_floor` and then by `word_id`. This verification confirmed that the final output was both complete and correctly ordered.

The output file is generated only after all words have been completed. The parent first creates an array of pointers to shared words and sorts it by:

1. `sorting_floor`,
2. `word_id`.

Then each line is written in the exact required format:

```
<word_id> <word> <sorting_floor>
```

No blank line is written. The arrival floor does not affect the final output. Therefore, the final file preserves the original word and sorting-floor information while enforcing the required ordering rule.

10 Test Case Scenario

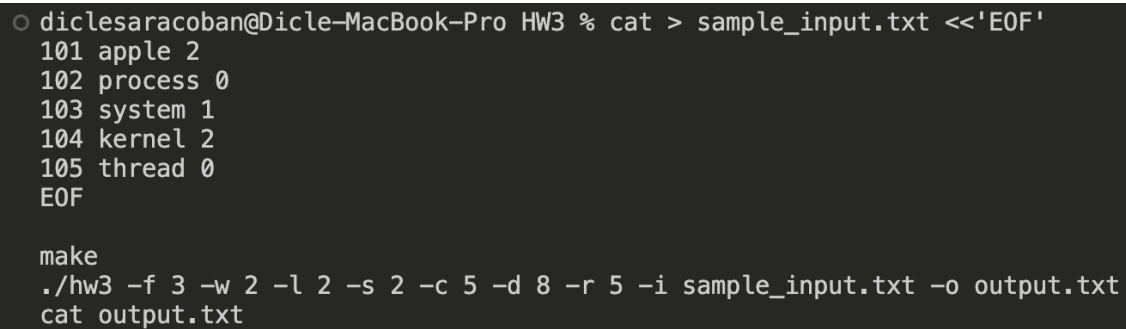
10.1 Mandatory Test Case

The produced output file is shown directly below the summary. It preserves all input words, keeps the original sorting-floor values, and is ordered by sorting floor and word id as required. The following test case was used to validate the basic functionality of the system.

Run command:

```
./hw3 -f 3 -w 2 -l 2 -s 2 -c 5 -d 8 -r 5 -i sample_input.txt -o output.txt
```

Input file screenshot:



```
o diclesaracoban@Dicle-MacBook-Pro HW3 % cat > sample_input.txt <<'EOF'
101 apple 2
102 process 0
103 system 1
104 kernel 2
105 thread 0
EOF

make
./hw3 -f 3 -w 2 -l 2 -s 2 -c 5 -d 8 -r 5 -i sample_input.txt -o output.txt
cat output.txt
```

Figure 1: Mandatory test case input file

Output / summary screenshot:

```
System Summary:
Total words: 5
Completed words: 5
Retries: 0
Characters transported: 30
Delivery elevator operations: 17
Reposition elevator operations: 151

≡ output.txt U X

HW3 > ≡ output.txt
      1      102 process 0
      2      105 thread 0
      3      103 system 1
      4      101 apple 2
      5      104 kernel 2
```

Figure 2: Mandatory test case summary and output

Observed behavior:

- The program initialized multiple process types on each floor.
- Words were admitted only when both arrival-floor and sorting-floor capacities were available.
- Characters were transported concurrently and reconstructed on their sorting floors.
- The final output file was produced in the required order.

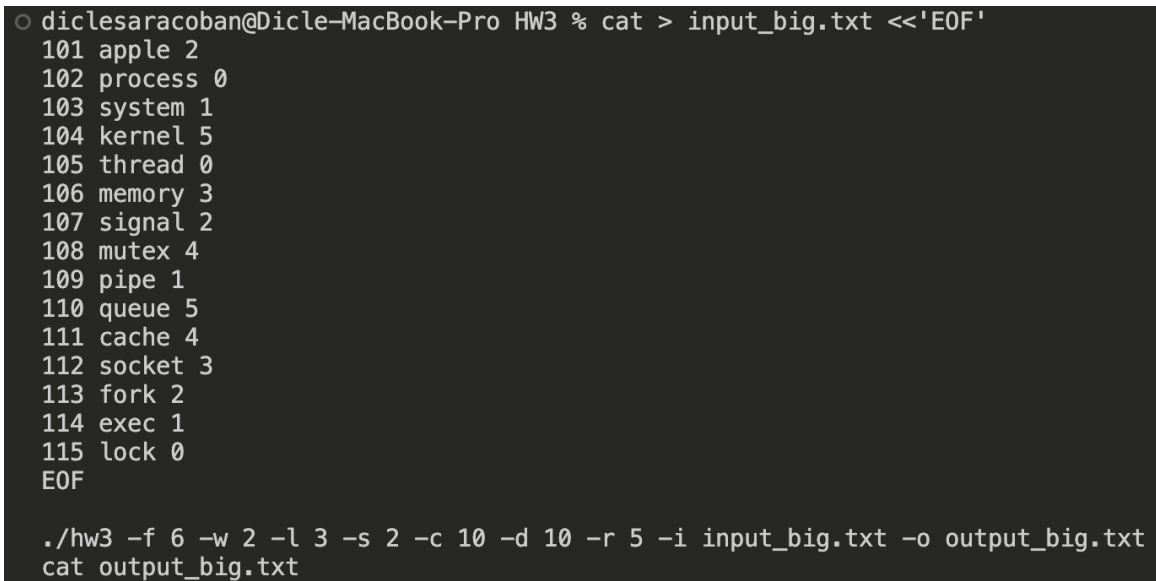
10.2 Large Input Test

This test was used to observe the behavior of the system under a larger workload.

Run command:

```
./hw3 -f 6 -w 2 -l 3 -s 2 -c 10 -d 10 -r 5 -i input_big.txt -o output_big.txt
```

Input file screenshot:



```
o diclesaracoban@Dicle-MacBook-Pro HW3 % cat > input_big.txt <<'EOF'
101 apple 2
102 process 0
103 system 1
104 kernel 5
105 thread 0
106 memory 3
107 signal 2
108 mutex 4
109 pipe 1
110 queue 5
111 cache 4
112 socket 3
113 fork 2
114 exec 1
115 lock 0
EOF

./hw3 -f 6 -w 2 -l 3 -s 2 -c 10 -d 10 -r 5 -i input_big.txt -o output_big.txt
cat output_big.txt
```

Figure 3: Large input test file

Output / summary screenshot:

```
Total words: 15
Completed words: 15
Retries: 12
Characters transported: 79
Delivery elevator operations: 62
Reposition elevator operations: 408
```

≡ *output_big.txt* U X

HW3 > ≡ output_big.txt

1	102	process	0
2	105	thread	0
3	115	lock	0
4	103	system	1
5	109	pipe	1
6	114	exec	1
7	101	apple	2
8	107	signal	2
9	113	fork	2
10	106	memory	3
11	112	socket	3
12	108	mutex	4
13	111	cache	4
14	104	kernel	5
15	110	queue	5
16			

Figure 4: Large input test summary and output

Observed behavior:

- The system completed all words successfully under a larger input size.
- The sorting order of the final output remained correct.
- Retry count increased compared to the basic case because more words competed for resources.

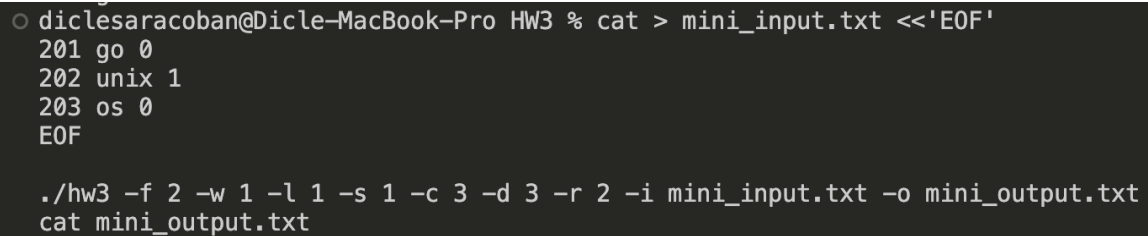
10.3 Mini Input Test

This smaller test case was used for quick verification of correctness.

Run command:

```
./hw3 -f 2 -w 1 -l 1 -s 1 -c 3 -d 3 -r 2 -i mini_input.txt -o mini_output.txt
```

Input file screenshot:



```
o diclesaracoban@Dicle-MacBook-Pro HW3 % cat > mini_input.txt <<'EOF'
201 go 0
202 unix 1
203 os 0
EOF

./hw3 -f 2 -w 1 -l 1 -s 1 -c 3 -d 3 -r 2 -i mini_input.txt -o mini_output.txt
cat mini_output.txt
```

Figure 5: Mini input test file

Output / summary screenshot:

```
System Summary:
Total words: 3
Completed words: 3
Retries: 0
Characters transported: 8
Delivery elevator operations: 4
Reposition elevator operations: 3998

≡ mini_output.txt U X

HW3 > ≡ mini_output.txt

1    201 go 0
2    203 os 0
3    202 unix 1
4
```

Figure 6: Mini input test summary and output

Observed behavior:

- The system handled a very small workload correctly.
- The final output order matched the expected sorting-floor and word-id ordering.
- This test was useful for quick debugging and sanity checking.

11 Challenges and Solutions

The implementation was challenging mainly because several process groups modify related shared state concurrently. The most important problems and solutions were:

Challenge	Solution
Duplicate word claims	Protected round-robin scan and claim operations with the global mutex.
Inconsistent admission state	Performed atomic dual-floor capacity checking with deterministic lock order.
Concurrent sorting on the same word	Added a per-word sorter lock in addition to the per-word mutex.

Duplicate character delivery risk	Used per-character claim, in-transit, delivered, and arrived flags under the word mutex.
Idle-carrier coordination	Added reposition requests and a dedicated reposition elevator queue.
Termination detection	Used completed flags and a completed-word counter monitored by the parent.
Cross-process wake-up of respawned carriers	Introduced carrier-specific named event semaphores that can be reopened safely after process creation.

12 Conclusion

This homework required the integration of process creation, shared memory, synchronization, queue-based subsystem coordination, and system-wide termination logic. The final design uses a shared-memory state model, per-word and per-floor synchronization, and separate delivery/reposition elevator queues to keep the system correct under concurrency.

The implementation satisfies the core assignment requirements: words are admitted atomically, characters are delivered exactly once, sorting is protected from concurrent corruption, the output file is generated after system completion, and the parent process performs coordinated shutdown and cleanup.